Programming Language Prolog

Prolog or **PRO**gramming in **LOG**ics is a fourth generation logical and declarative programming language that is suitable for writing programs that involve symbolic or non-numeric computation.

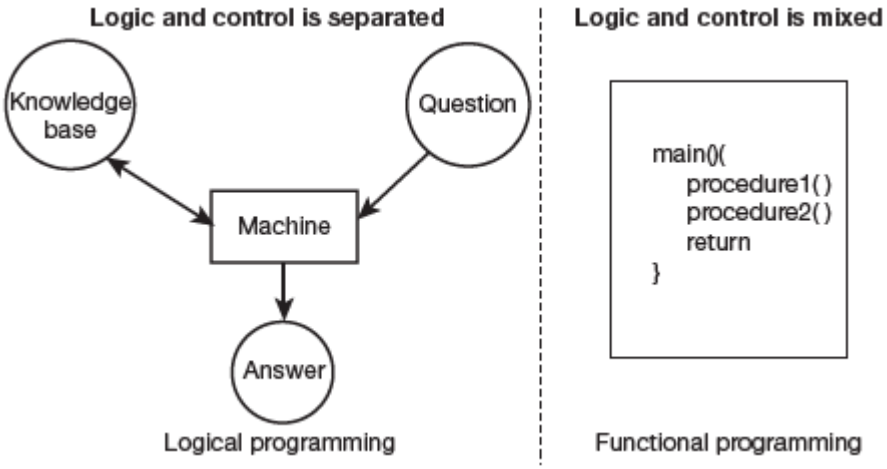
In Prolog, statements in the program are used to express the facts and rules about different problems within a system of formal logic.

The rules are written as logical clauses having a head and a body. For example, if H is head and B1, B2, B3 are the elements of the body, then “H is true, when B1, B2, B3 all are true”.

You can consider facts as rules that are without any body. For example, “H is true”.

Logic and Functional Programming

We have already seen the difference between a Logic programming language and a traditional functional programming language. Figure 1 summarizes this difference pictorially.



While functional programming requires step-by-step instructions to solve a particular problem, logic programming, on the other hand, uses a

knowledge base to find answers to the given questions.

In functional programming, we have to specify the steps to solve a particular problem, but in logic programming, we just have to give the problem. The logic programming automatically finds a suitable solution to solve that problem.

What is Prolog?

To solve a problem in Prolog, we need not specify the following three things:

1. **Facts:** A fact is a predicate that is true, for example, if we say, “Tom is a dog”.
2. **Rules:** A rule contains facts that has conditional clauses. To satisfy a rule these conditions should be satisfied. For example, if there is a rule, grandfather(X, Y):- father(X, Z), parent(Z, Y).
   This means that X is the grandfather of Y if Z is a parent of Y and X is the father of Z.
3. **Questions:** To run a Prolog program, we need some questions that are answered using the given facts and rules.

Applications of Prolog

Prolog is used in different areas to automate several tasks including,

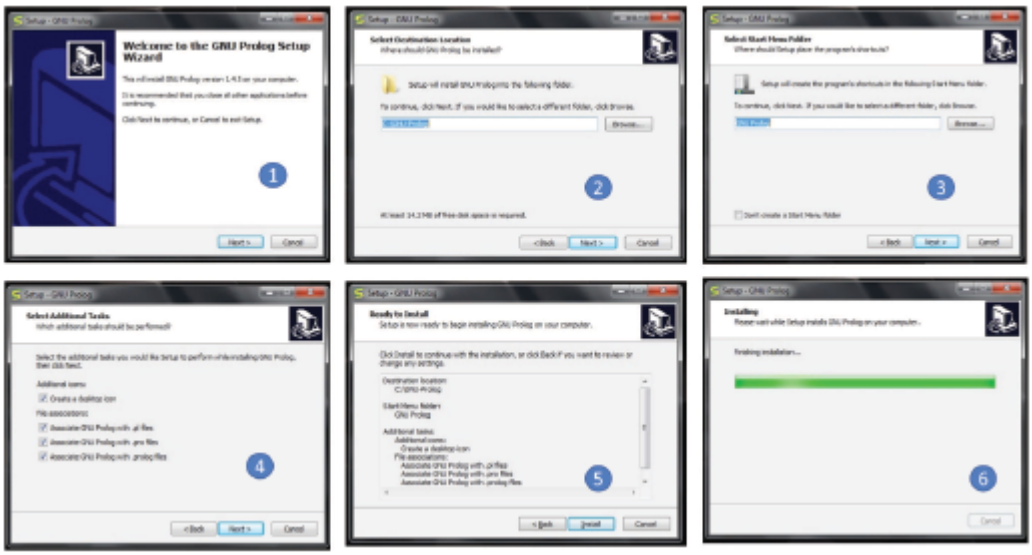
- Intelligent database retrieval
- Natural language understanding
- Specification language
- Machine learning
- Robot planning
- Automation system
- Problem solving

Installing Prolog

In this section, we have used GNU Prolog version 1.5. You can download it from the official GNU Prolog website, **http://www.gprolog.org/** or you can straight away go the download link for Windows by using the link,

http://www.gprolog.org/setup-gprolog-1.4.5-mingw-x64.exe(64 Bit System).

On clicking this link, you will get some dialog boxes, just press NEXT buttons and FINISH button at the end. Prolog will be installed in your computer.

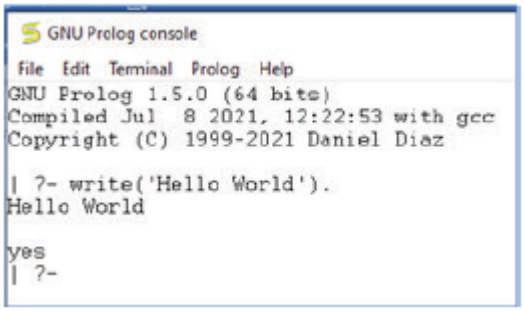


Credit: Copyright (C) 1999-2021 Daniel Diaz

Hello World Program

Let's write the first program to print Hello World in Prolog.

Start the GNU Prolog and write the command as follows:



Credit: Copyright (C) 1999-2021 Daniel Diaz

write('Hello World')

Note that after each line, put one period (.) symbol to show that the line has ended.

The corresponding output will be as shown in figure.

We can put these instructions in a file by writing the instruction(s) in a file and then saving it with a .pl extension. However, before executing the .pl file, make sure that either this file is stored in the directory where the GNU Prolog console is pointing or the directory should be changed to do so. To change the directory, follow the steps given below.

Step 1: From the Prolog console, go to File -> Change Dir, then click on that menu.

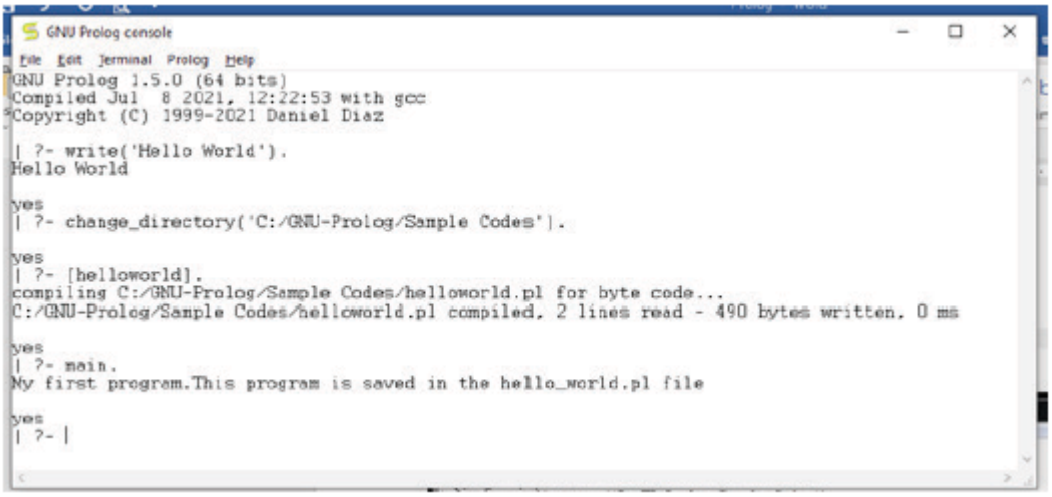
Step 2: Select the proper folder and press OK.

Once this is done, write the following instructions a .pl file and run the file in Prolog console as shown below.

main:- write('My first program.'),

write('This program is saved in the hello_world.pl file').

We have saved the file as helloworld.pl in SampleCodes directory. We had created this directory in C:\GNU-Prolog directory. Note that we had changed the directory and also remember that filenames should be in small case.



Credit: Copyright (C) 1999-2021 Daniel Diaz

Creating Facts and Rules

In this section, we will focus on creating the knowledge base for problem solving. This KB is the fundamental part of Logic Programming and contains facts, rules and questions/queries.

Facts represent an explicit relationship between objects, and properties. They are unconditionally true in nature. For example,

Tom is a dog	Ria loves to eat icecream	Jia stood first.
Stars twinkle	Sun is hot	Pratyusha plays cricket

To write facts in Prolog, we must remember the following points:

- Names of properties/relationships begin with lower case letters.
- Name of the relationship is written as the first term.
- Objects appear as comma-separated arguments within parentheses.
- A fact must be ended with a period “.”.

- Objects begin with lower case letters. They can also start with digits, and can be strings of characters enclosed in quotes, e.g., color(hair, ‘black’).
- Phone_no(anuj, 1234567890). is a predicate or clause.

The syntax for defining facts is,

```
relation(object1,object2...).
```

For example,

```
dog(tom).          loves_to_eat(ria,icecream).    stood_first(jiya)
twinkle(stars).    hot(sun)                plays(pratyusha, cricket).
```

Rules define an implicit relationship between objects. It considers facts to be conditionally true. That is, when an associated condition is true, then the predicate is also true. For example,

- Mina is happy if she sings.
- Krish is hungry if he is eating stale food.
- Parul and Disha are friends if both of them go for dancing classes.
- Will go to play if it is a holiday and friend is free.

In case of a rule, if the right-hand side is true, then the left-hand side is also true. Technically, it is represented with the symbol (:-) and is pronounced as “If”, or “is implied by”. This is also known as neck symbol. In a rule, the LHS of this symbol is called the Head. Correspondingly, RHS is called Body.

To represent conjunction in a rule, we use comma (,) and to represent disjunction, semicolon is used.

Syntax of writing a rule is,

```
rule_name(object1, object2, ...) :- fact(object1, object2, ...)
```

If we have a rule as, P:- Q;R. Then, it can be written as, P:- Q. P:- R.	If a rule is, P:- Q, R; S, T, U. Then, it can also be written as, P:- Q, R. P:- S, T, U.

Example

```
happy(mina) :- sings(mina).
hungry(krish) :- eating_stale_food(krish).
friends(parul,disha) :- danceClass(parul), danceClass(disha).
playGuitar(manav) :- holiday(school), free(manav).
```

Queries are some questions on the relationships between objects and object properties. Logic programming language finds answers to these questions and returns them. For example, as per our facts and rules, some queries can be,

Is tom a dog?	Does riya loves to eat iceccream?
Is mina happy?	Will Manav play guitar?

Creating Knowledge Base 1 Containing Facts in Prolog

Open a file in any text editor (like WordPad, NotePad, etc.) and write the facts given below in it. Save this file with any name and .pl extension. We have saved this file as kb1.pl in the SampleDocs folder.

student(ben).

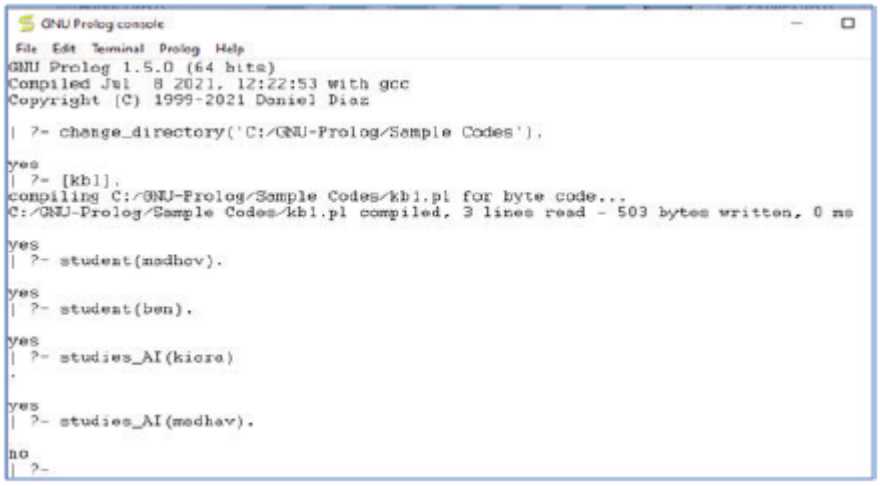
student(kiara).

student(madhav).

studies_AI(kiara).

Now, in the GNU-Prolog, we will ask queries and get a reply as shown in the figure given below. Note that every time we start GNU-Prolog, we have to change directory to set it where our files are stored. Then we have to compile our knowledge base before executing the queries.

Names of persons, filenames, objects and relations are all written in lowercase letters, because in Prolog, a string starting with uppercase letter indicates a variable.

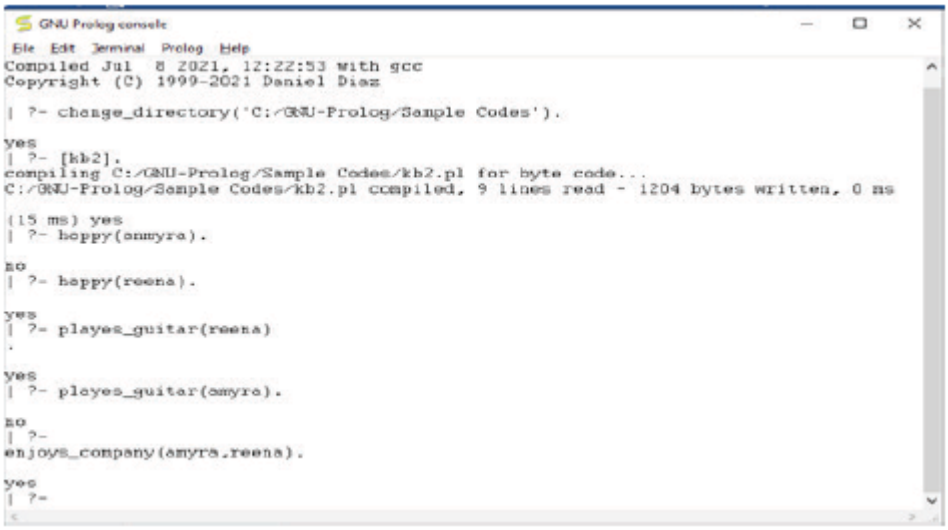


Credit: Copyright (C) 1999-2021 Daniel Diaz

Creating Knowledge Base 2 Containing Facts and Rules in Prolog

Open a new file in text editor and write the following facts and rules in it. Save this file as kb2.pl in SampleDocs folder.

```
dances(amyra).
sing_song(reena).
sing_song(amyra) :- dances(amyra).
happy(amyra) :- dances(amyra).
happy(reena) :- sing_song(reena).
playes_guitar(reena) :- sing_song(reena).
enjoys_company(amyra,reena) :- sing_song(reena).
```

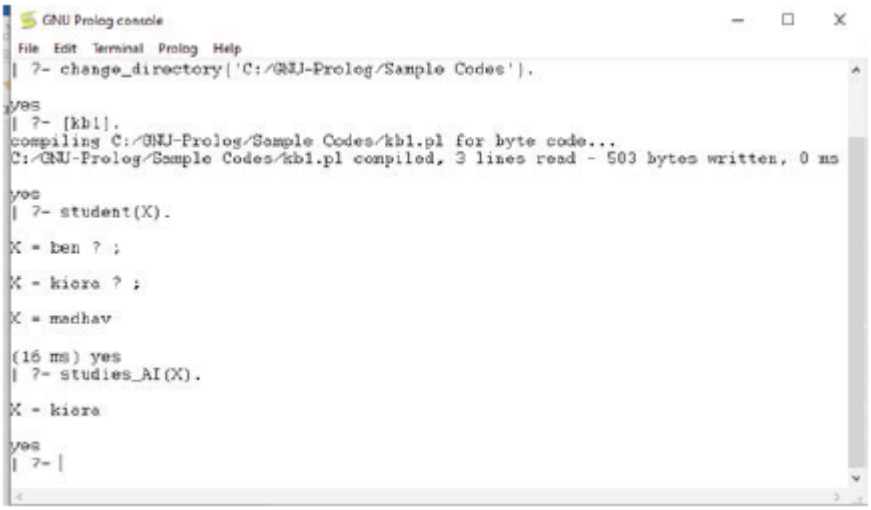


Credit: Copyright (C) 1999-2021 Daniel Diaz

Using Variables

We will use knowledge base 1 (kb1.pl) to demonstrate working with variables. Remember that, name of the variable starts with a capital letter. For example, if we want to know who all the students are, then we can write, student(X).

In the result, names of the students will be displayed one by one. Press the Enter key or semicolon (;). While pressing Enter key will just display a single result, pressing a semi-colon will display the next result. Look at the figure to understand this concept.



Credit: Copyright (C) 1999-2021 Daniel Diaz

Working with Relationships in Prolog

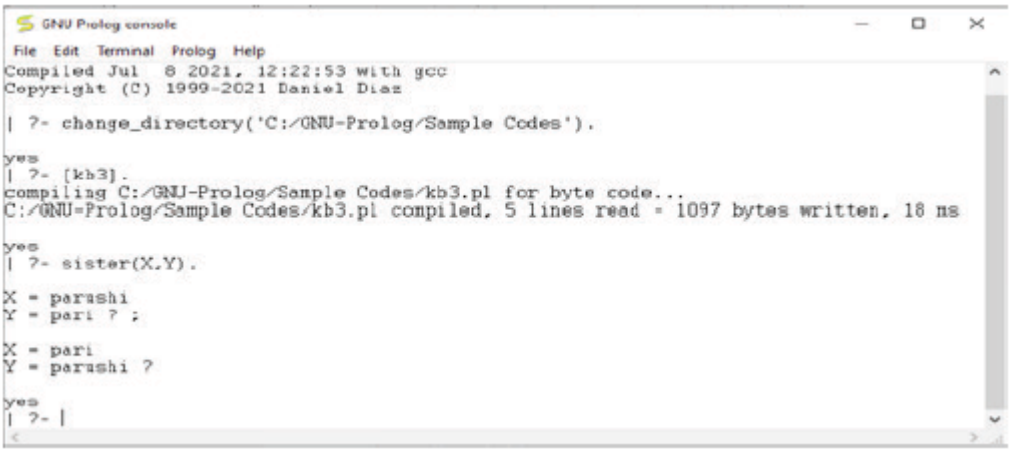
We can easily express relationships in Prolog. These relationships exist between objects and their properties. For example, if we say, Tina has a car, then, it means Tina is the owner of a car. Here, owner is the relationship that exist between two objects- Tina and car.

When we pose a query, “Does Tina own a car?”, we are actually trying to find out whether this relationship exists or not. Some relationships are expressed using rules. A rule can find out about a relationship even if it is not defined explicitly as a fact.

Create a new file kb3.pl and write the following facts and rules in it. These are used to find whether two persons are sisters to each other or not.

```
parent(sanat, parushi).
parent(sanat, pari).
female(parushi).
female(pari).
sister(X,Y) :- parent(Z,X), parent(Z,Y),female(X), female(Y), X\==Y.
```

Now, open GNU-Prolog and find out between which two individuals this relationship exists.



Credit: Copyright (C) 1999-2021 Daniel Diaz

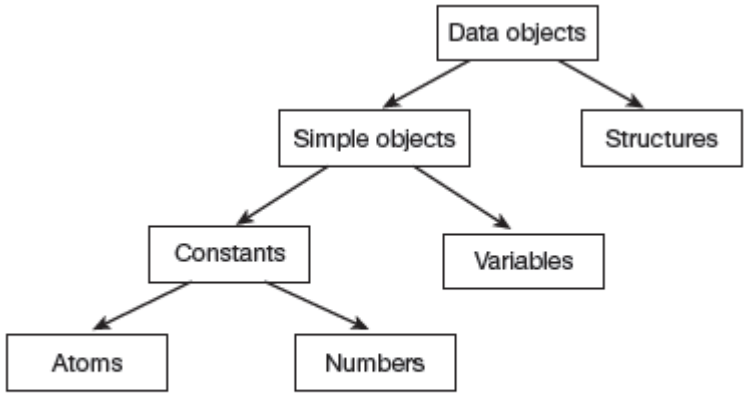
You can add information about more individuals int it. You can even define more relationships. For example,

- mother(X,Y):- parent(X,Y), female(X).
- brother(X,Y):- parent(Z,X), parent(Z,Y), male(X), X \== Y.
- haschild(X):- parent(X,_).
- grandparent(X,Y):- parent(X,Z), parent(Z,Y).
- grandmother(X,Z):- mother(X,Y), parent(Y,Z).
- grandfather(X,Z):- father(X,Y), parent(Y,Z).
- wife(X,Y):- parent(X,Z),parent(Y,Z), female(X),male(Y).
- uncle(X,Z):- brother(X,Y), parent(Y,Z).

In Prolog we can trace the execution to see how output was obtained. To enter into the trace mode, type “trace” in the Prolog console.

Data Objects

In Prolog, we can divide data objects into the following categories.



DATA objects in prolog

Below are some examples of different kinds of data objects:

- Atoms –
- Numbers – 100, 1235, 2000.45
- Variables – X, Y, Xval, _X
- Structures – day(9, jun, 2017), point(10, 25)

Atoms: Atoms are like constants. They can be any names or objects. For example, amyra, num_of_digits, c3, etc. While using atoms, we must consider the following points:

- An atom can be a string of alphabets, digits and the underscore character, '_'.
- It should start with a lower-case letter.
- An atom can be a string of special characters that already have a predefined meaning. For example, :-' <--->, =======>, ..., .:., ::=
- An atom can be a string of characters enclosed in single quotes. This is important when we have an atom that starts with a capital letter. By enclosing it in quotes, the string becomes easily distinguishable from variables. For example, 'Delhi', 'Narendra_Modi.

Numbers: They represent values that can be integers like 100, 4, -81, 2022. In Prolog, range of integers is from -16383 to 16383. Numbers can also be real numbers, but floating-point numbers are rarely used in Prolog programs as Prolog is usually used for symbolic, non-numeric computation. For example, 3.14159, -0.00029, 123.45, etc., are real numbers.

Defining Variables in Prolog

Like every programming language, Prolog also lays some rules to define new variables. These rules states,

- Variables are strings of letters, digits and underscore characters.
- They start with an upper-case letter or an underscore character.

Examples of valid names of variables are X, Val, Student_name, Shoppinglist, _a50, _39.

Anonymous Variables in Prolog

Anonymous variables have no names. They are written using a single underscore character '_'. And each anonymous variable is treated as different. They are not same. These variables are very useful in certain situations. For example, if we have facts:

"John plays cricket" and "Shivansh plays tennis", then we can easily find out who plays cricket using an ordinary variable. But, if the task is to find whether anyone plays cricket. Here, the name of the person who plays cricket is not required, so in such a case we must use anonymous variables.

In general, anonymous variables are used when there is a need to use a variable without knowing its value. Look at the code given below that demonstrates the use of anonymous variables. Here, we have used knowledge base 1 to find whether we have any student in the KB. Note that the console replies only 'yes' or 'no', I does not show names (or values) of the students.



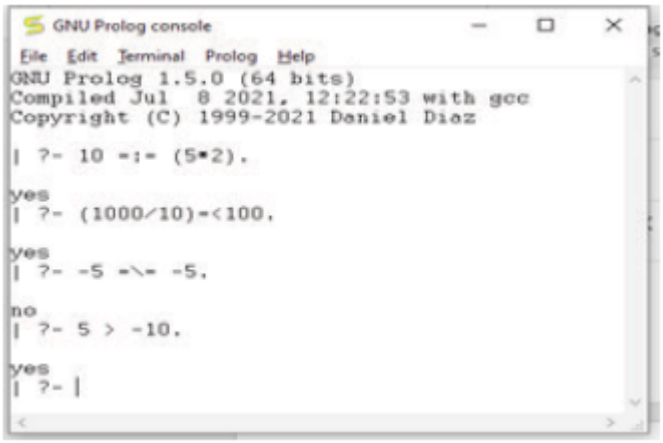
Credit: Copyright (C) 1999-2021 Daniel Diaz

Comparison Operators

Comparison operators are used to compare two equations or states. Some comparison operators are listed in Table.

Operator	Meaning
X > Y	X is greater than Y
X < Y	X is less than Y
X >= Y	X is greater than or equal to Y
X =< Y	X is less than or equal to Y
X =:= Y	Values of X and Y are equal
X =\= Y	Values of X and Y values are not equal

Let us try these operators in GNU-Prolog now.



Credit: Copyright (C) 1999-2021 Daniel Diaz

Arithmetic Operators in Prolog

Arithmetic operators are used to perform arithmetic operations. Some popularly used arithmetic operators are given in Table. To implement these operators, create a new file and save it as ao.pl. Write the following code in it.

```
calc :- X is 10 + 20, write('10 + 20 is '),write(X),nl,

Y is 30 - 15, write('30 - 15 is '),write(Y),nl,

Z is 10 * 5, write('10 * 5 is '),write(Z),nl,

A is 200 / 40, write('200 / 40 is '),write(A),nl,

B is 250 // 17, write('250 // 17 is '),write(B),nl,

C is 50 ** 2, write('50 ** 2 is '),write(C),nl,

D is 220 mod 40, write('220 mod 40 is '),write(D),nl.
```

Now, run this code in the console as shown below. Note that nl means new line.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Power
//	Integer Division
mod	Modulus



Credit: Copyright (C) 1999-2021 Daniel Diaz

Loops

Loop statements are used to execute one or more instructions multiple times. A set of instructions is executed in Prolog multiple times using recursive predicate logic. Unlike, C, C++, Java, there are no direct loops in Prolog. To simulate loop in Prolog, create a file count10.pl, write the instructions given below and execute it as shown below.

```
count_to_10(L) :-
   between(L, 10, Y),
   Z is L + Y,
      write(Z), nl.
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Let us write another program to count down from 10 to 0. Create a new file, write the instructions given below and save it as count_down_10.pl.

```
count_down_to_10(L):-
        between(L,10,Y),
        Z is 10-Y,
        write(Z),nl.
```

Execute the file as shown below.



Credit: Copyright (C) 1999-2021 Daniel Diaz

Decision Making

Like in other programming languages, decision statements in Prolog are If-Then-Else statements. We check a condition. If it is true, then a task is performed and if it false, another set of tasks is performed.

The syntax to use a decision statement in Prolog is,

```
If <condition> is true, Then <do this>, Else
```

To implement decision making, create a new file, write the instructions given below and save it as dm.pl.

```
gte(X,Y) :- X > Y,write('X is greater than Y').
gte(X,Y) :- X =:= Y,write('X is equal to Y').
gte(X,Y) :- X < Y,write('X is less than Y').
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Conjunction

Conjunction, commonly known as AND logic, is implemented using the comma (,) operator. That is, two predicates separated by comma are joined with AND statement. For example, if we have two predicates,

```
parent(nina,kimaya) indicating nina is the parent of kimaya
female(nina) specifying that nina is a female
```

We can write the rule as,

```
mother(nina,kimaya) :- parent(nina,kimaya), female(nina)
```

The rule states that nina is the mother of kimaya if nina is the female parent of kimaya.

Disjunction

Disjunction, commonly known as OR logic is implemented using the semicolon (;) operator. This means that two predicates separated by semicolon are joined with OR statement. For example, if we have two predicates

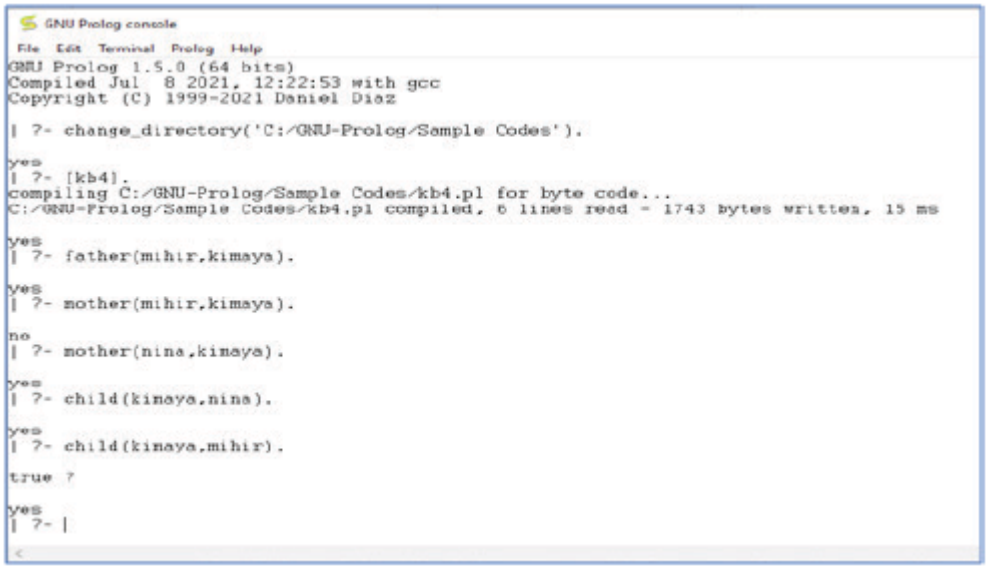
```
father(mihir,kimaya) indicating mihir is father of kimaya
mother(nina,kimaya) specifying nina is mother of kimaya
```

Then, we can write the rule,

```
child(X,kimaya) :- father(X,kimaya); mother(X,kimaya)
```

This means that kimaya is the child of X if X is father or mother of kimaya,

```
parent(nina,kimaya).
parent(mihir,kimaya).
female(nina).
male(mihir).
mother(nina,kimaya) :- parent(nina,kimaya),female(nina).
father(mihir,kimaya) :- parent(mihir,kimaya),male(mihir).
child(kimaya,X) :- father(X,kimaya);mother(X,kimaya).
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Lists in Prolog

List is a simple data structure that can contain any number of items. Individual items of a list are placed within square brackets ([]). For example, a list of students, we will write, [‘Manya’, ‘Amyra’,’Raman’,’John’,’Zeenat’]. We can also define an empty list by writing, a Prolog atom, [].

A list with element(s) have two important things,

- Head which is the first item of the list
- Tail that is the remaining part of the list

In the list, [‘Manya’,’Amyra’,’Raman’,’John’,’Zeenat’], ‘Manya’ is the head of the list. Tail is another list consisting of [‘Amyra’,’Raman’,’John’,’Zeenat’].

Note that if we have a list, L = [a, b, c]. Then, Tail = [b, c]

This can be also written as L = [a | Tail].

Here, the vertical bar (|) separates the head and tail parts.

List can also be represented as,

- [a, b, c] = [a, | [b, c]]
- [a, b, c] = [a, b | [c]]
- [a, b, c] = [a, b, c | [ ]]

Basic Operations on Lists

Table lists some operations that can be performed on lists in Prolog.

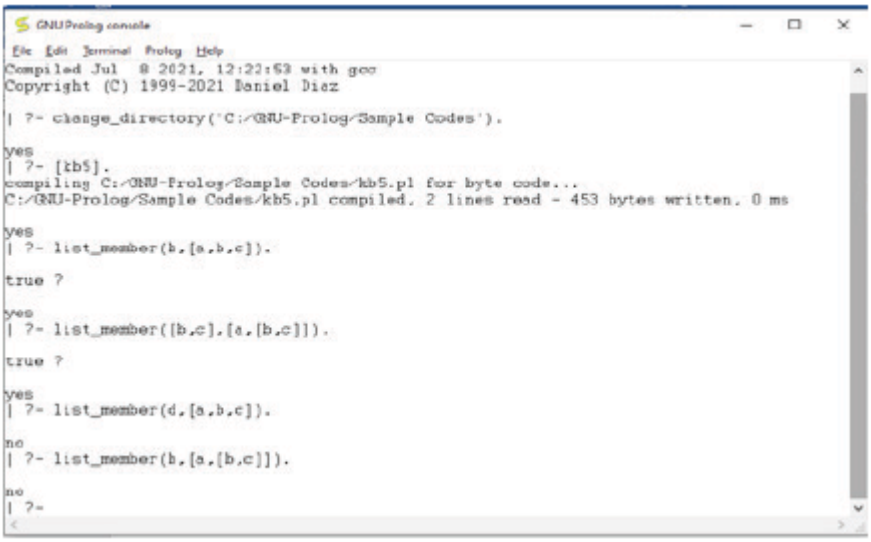
Operations	Definition
Checking Membership	To check if a given element is member of the specified list
Calculate Length	Find the length of a list.
Concatenation	Join/add two lists.
Delete Items	Removes the specified element from a list.
Append Items	Adds the element at the head of a list.
Insert Items	Adds an element at the specified position in the list.

Membership Operation

To check membership, we define a predicate, list_member(X,L) that will consider two things. If X is a member of a list, then either X is head of the list or is a member of the tail of the list.

To implement this predicate, create a new file in text editor, write the following lines of code in it and save it as kb5.pl.

```
list_member(X,[X|_]).
list_member(X,[_|TAIL]) :- list_member(X,TAIL).
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Length Calculation

This is used to find the length of list L. We will define one predicate to do this task. Suppose the predicate name is list_length(L,N). This takes L and N as input argument. This will count the elements in a list L and instantiate N to their number. As was the case with our previous relations involving lists, it is useful to consider two cases –

- If list is empty, then length is 0.
- If the list is not empty, then L = [Head|Tail], then its length is 1 + length of Tail.

To find length of a list in Prolog, add the lines given below in kb5.pl file and then execute as shown in Figure.

```
list_length([],0).
list_length([_|TAIL],N) :- list_length(TAIL,N1), N is N1 + 1.
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Concatenation

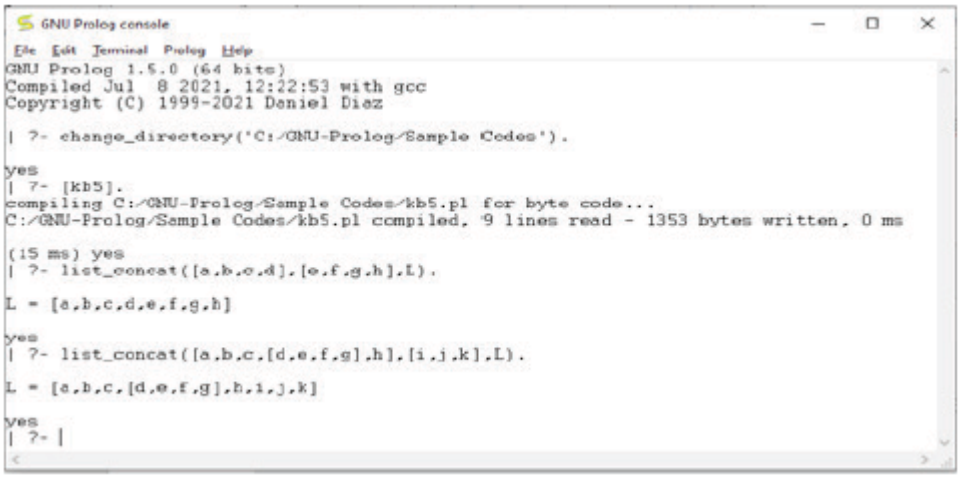
Concatenating two lists' means adding the elements of the second list after the first one. For example, if we have a list with elements [a,b,c,d] and another list with elements [e,f,g,h], then the concatenated list will have elements [a,b,c,d,e,f,g,h].

To implement list_concat predicate in Prolog, add the following lines in kb5.pl file and execute the instructions shown in Figure.

Remember that,

- **`list_concat()`** takes first list, second list, and the concatenated. There are two observations here.
- If the first list is empty, and second list is L, then the concatenated list will be L.
- If the first list is not empty, then [Head|Tail] of first list, concatenate Tail with second list recursively, and store into new list in the form, [Head|New List].

```
list_concat([],L,L).
list_concat([X1|L1],L2,[X1|L3]) :- list_concat(L1,L2,L3).
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

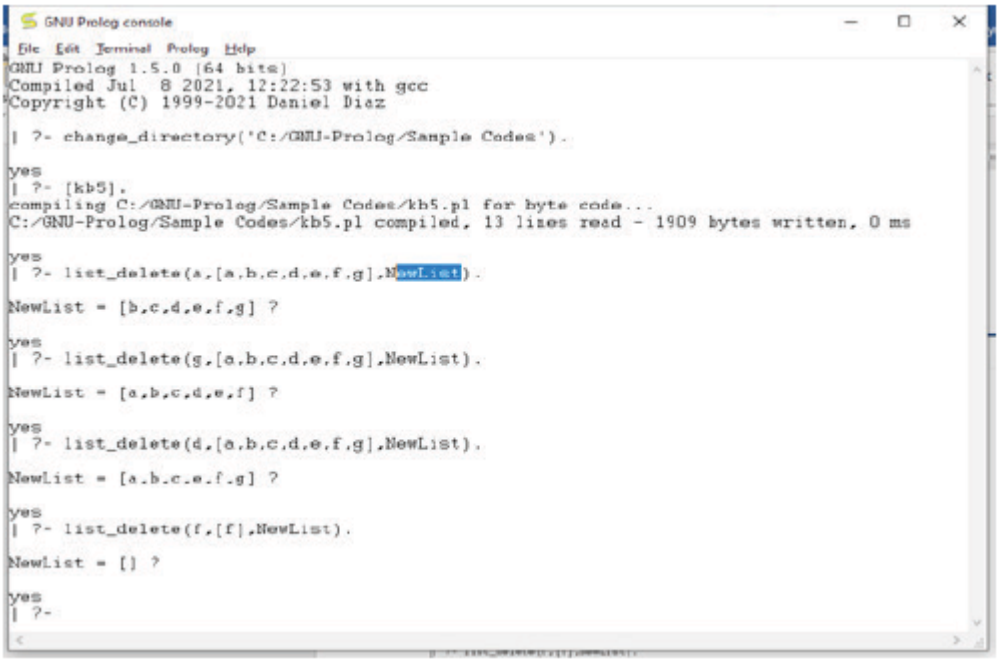
Delete from List

To delete an element X from a list L, one of the three cases may arise.

- If X is the only element, then after deleting it, an empty list is returned.
- If X is head of L, then the Tail part of the list will be returned.
- If X is an element of the Tail of the list then recursively delete from the tail part.

We need to add the following instructions for the predicate list_delete in the kb5.pl and run the commands as shown in figure.

```
list_delete(X, [X], []).
list_delete(X,[X|L1], L1).
list_delete(X, [Y|L2], [Y|L1]) :- list_delete(X,L2,L1).
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Append into List

While appending two lists, we add two lists or add one list as an element of the other list. If the element to be added is already present in the list, then the append function simply returns.

Before writing instructions to implement list_append(L1, L2, L3) predicate, note that,

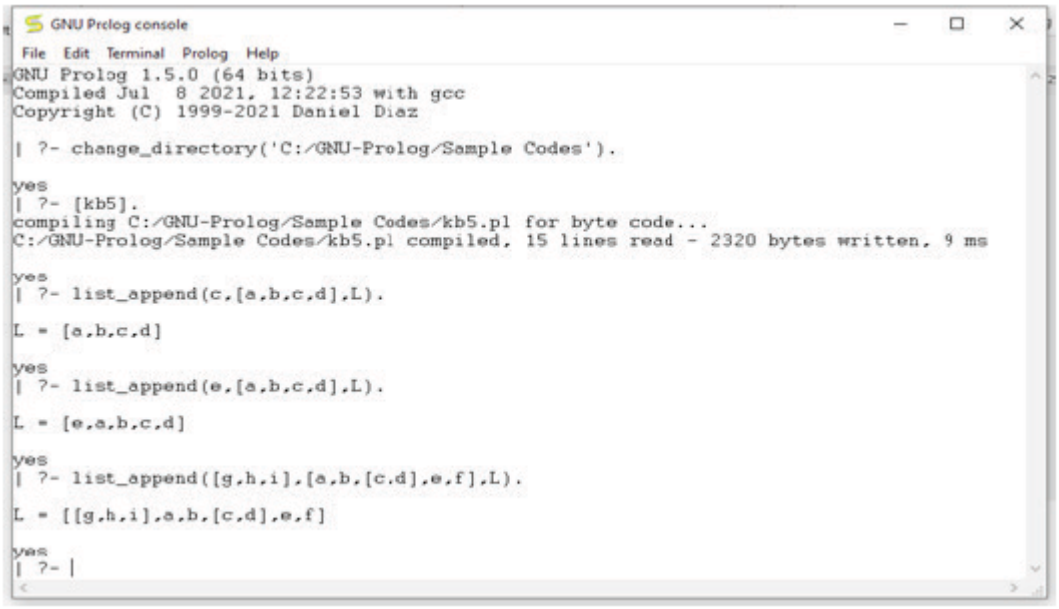
If A is an element to be appended in list L1 that already has A, then the output will be L1 only.

Otherwise, a new list will be created as, L2 = [A|L1].

To implement list_append predicate in Prolog, add the following statements in kb5.pl and execute the instructions in console as given in figure.

```
list_append(A,T,T) :- list_member(A,T),!.
list_append(A,T,[A|T]).
```

Note that the (!) symbol, is known as cut. Here it is used because we want that if the first line is executed successfully, then we cut it, so that the next operation is not executed.



Credit: Copyright (C) 1999-2021 Daniel Diaz

Reverse Operation

The reverse operation arranges the items of a list in reverse order. For example, a list L = [a,b,c,d,e] when reversed will give output as [e,d,c,b,a]. To create a predicate, list_reverse(List, ReversedList), we must remember that,

- If the input list is empty, then the reversed list will also be empty.
- In the input list, elements are organized as [Head | Tail], but in the reversed list, elements in the Tail are recursively concatenated with the Head.

Add the instructions to implement list_rev predicate in kb5.pl and check the execution in console as shown in figure.

```
list_rev([],[]).
list_rev([Head|Tail],Reversed) :-
   list_rev(Tail, RevTail),list_concat(RevTail, [Head],Reversed).
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Shift Operation

Shift operation is used to shift one element of a list to the left rotationally. For example, given a list [a,b,c,d], then after shifting, it will be [b,c,d,a]. To implement shifting operation in Prolog, copy the following rule in kb5.pl and execute the instructions as given below. The clause list_shift(L1, L2) takes an input list in the form [Head|Tail] and then recursively concatenates Head after the Tail.

```
list_shift([Head|Tail],Shifted) :- list_concat(Tail, [Head],Shifted).
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

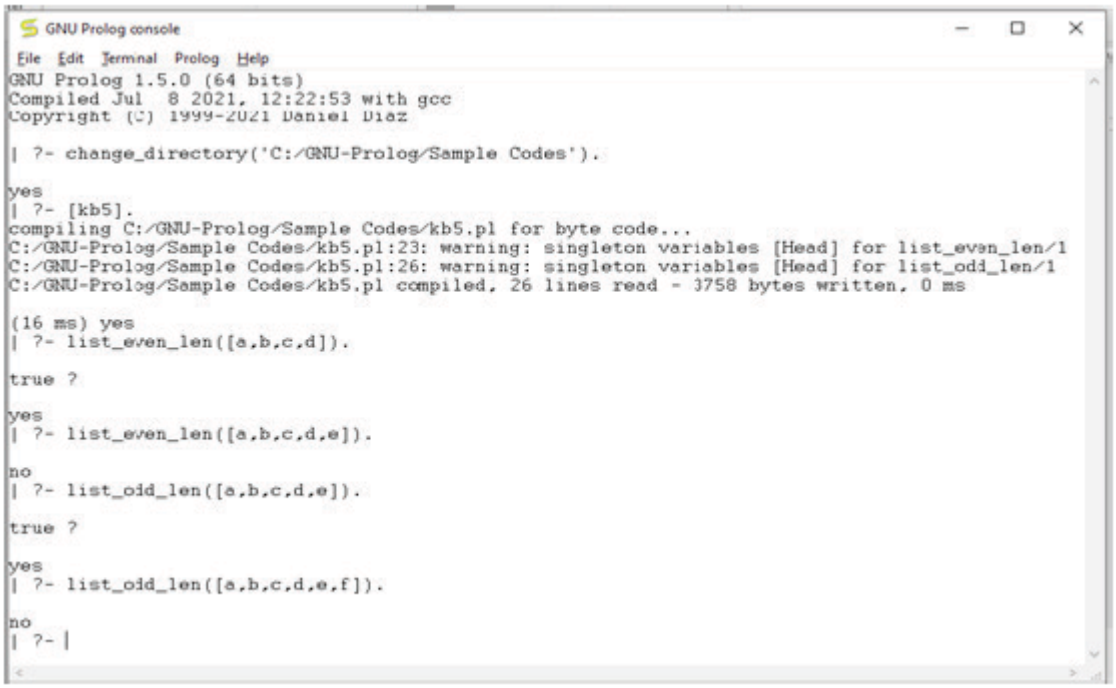
Miscellaneous Programs

Program 1: Check whether the list has odd number of elements or the even number of elements.

To implement this functionality, we will define predicates list_even_len(L) and list_odd_len(L) while considering the following points.

- An empty list has zero length. Thus, even length list.
- A list with a single element is an odd length list.
- Otherwise, a list [Head | Tail], has odd length if Tail is even length.
- A list [Head | Tail], has even length if Tail is odd length.

Write the following rules in kb5.pl and execute the commands as shown below.



Credit: Copyright (C) 1999-2021 Daniel Diaz

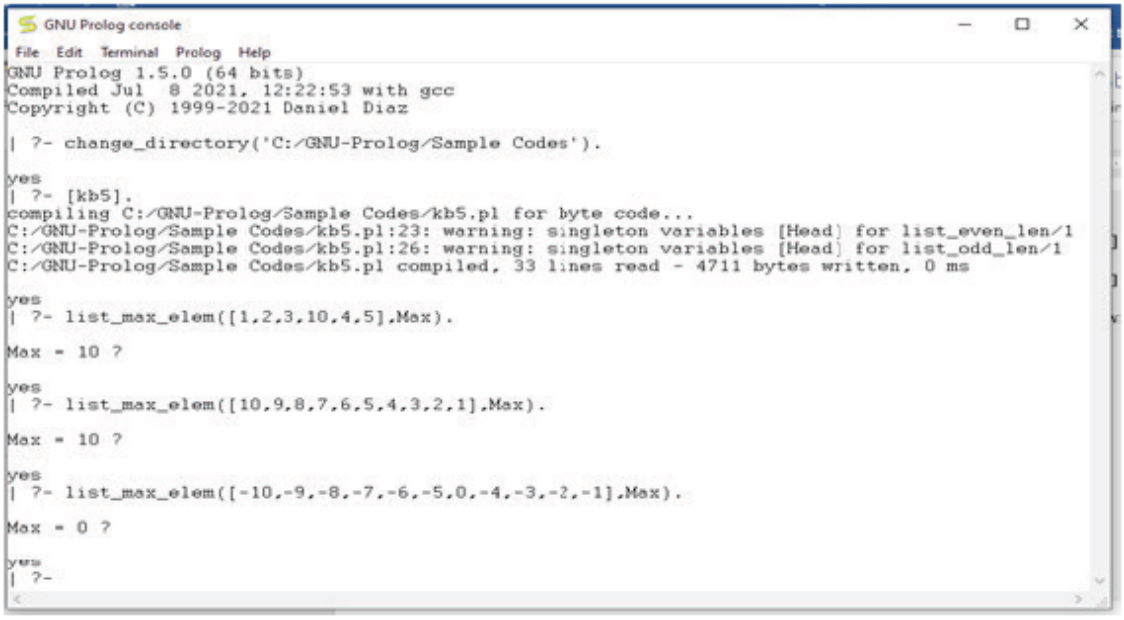
Program 2: Find the maximum element from a list.

Let us define a predicate, list_max_elem(List, Max) considering that,

- If there is only one element in the list, then Max will be that element itself.
- Else, divide the list as [X,Y|Tail]. Recursively find Max of [Y|Tail] and store it into MaxRest. Similarly, find maximum of X and MaxRest, and store it in Max.

To execute this predicate, first write the following rules in kb5.pl and then run the commands in console as shown below.

```
max_of_two(X,Y,X) :- X >= Y.
max_of_two(X,Y,Y) :- X < Y.
list_max_elem([X],X).
list_max_elem([X,Y|Rest],Max) :-
   list_max_elem([Y|Rest],MaxRest),
   max_of_two(X,MaxRest,Max).
```



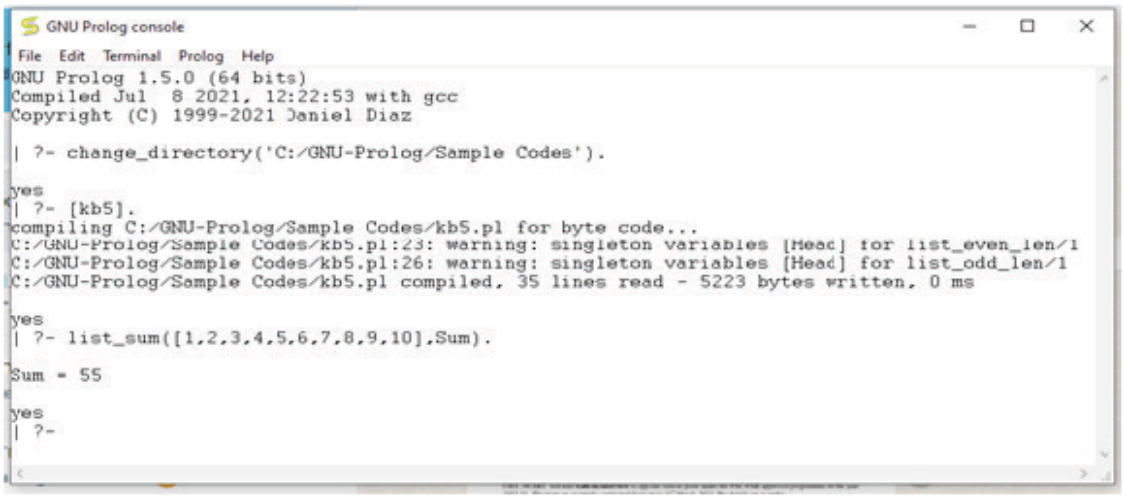
Credit: Copyright (C) 1999-2021 Daniel Diaz

Program 3: Find the sum of elements of a list.

Here, we will define a predicate, list_sum(List, Sum). If the list is empty, then sum will be 0. Else, list in the form of [Head|Tail], recursively finds sum of the tail and store them into SumTemp. Finally, sum is calculated as, Sum = Head + SumTemp.

For this, write the following rules in kb5.pl and execute the predicate as shown below.

```
list_sum([],0).
list_sum([Head|Tail], Sum) :- list_sum(Tail,SumTemp), Sum is Head +
SumTemp.
```



Credit: Copyright (C) 1999-2021 Daniel Diaz

Handling Input and Output

Till now, we have executed queries on the console. But many-a-times, it is necessary to some information from the user, process it and then display the result on the console. For such applications, we must know how to perform input-output operations in Prolog.

The `write()` Predicate

As the name suggests, the **`write()`** predicate takes the parameter as input, and writes the content into the console (by default). The content can also be written in files. Look at figure which demonstrates the usage of **`write()`** predicate.

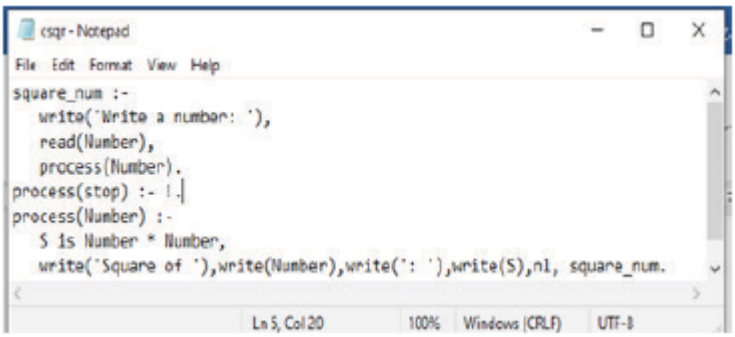
Note that we can insert a new line in the output displayed by writing 'nl'. From the figure, it is clear that to print a string on the console, we have to use single quotes ('string'). If we use double quote ("string"), then it will return a list of ASCII values.

The `read()` Predicate

The **`read()`** predicate is used to read some information from the user through the console. Users can type some content in the console, that can be taken as input and further processed. However, it is not mandatory

that Prolog programs can read only from console. The programs can also read data from files.

Figure shows a simple program that reads a number from the user and displays it square.

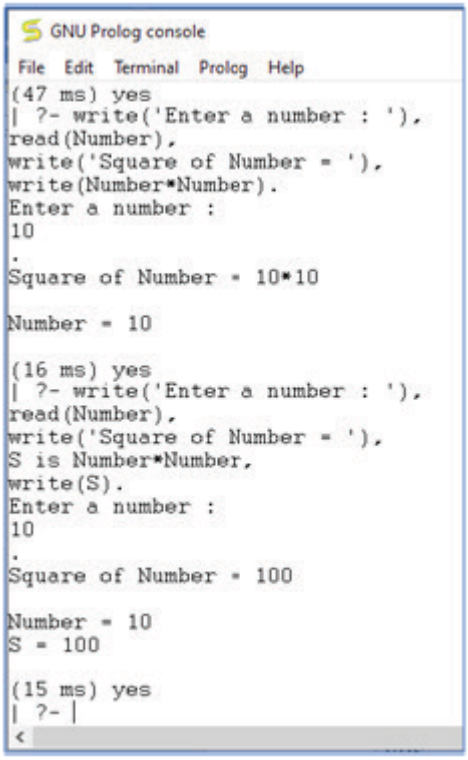


Credit: Copyright (C) 1999-2021 Daniel Diaz

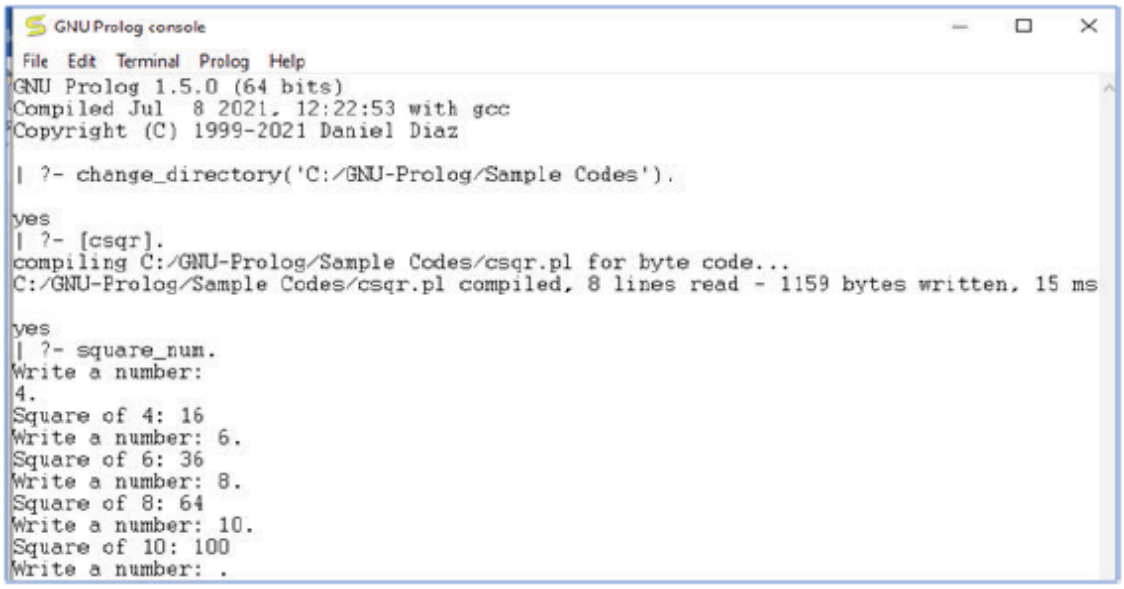
In Figure, we have extended this functionality by writing a program in a .pl file and then executing it in console. The program keeps reading number from user and displays its square until the user enters a '.' (full stop).



Credit: Copyright (C) 1999-2021 Daniel Diaz



Credit: Copyright (C) 1999-2021 Daniel Diaz



Credit: Copyright (C) 1999-2021 Daniel Diaz

The `tab()` Predicate

The **`tab()`** predicate is used to put some blank-spaces while we displaying some content. The predicate takes a number as an argument and prints those many numbers of blank spaces. For example, to print 5 blank spaces between Good and Morning write the following lines.

```
| ?- write('Good'),tab(5),write('Morning').
Good     Morning
```

We can open more than one file using tell(). When told is called, all files will be closed.

Reading/Writing Files

Prolog has some built-in predicates, that can be used to read from file and write into it. In this section, we will read about them.

The tell and told

The **`tell()`** predicate is used to write data into a file and not the console. For this, it accepts filename as the argument. If that file is not present, then a new file is created data is written into it. If the file already exists, it

will be opened for writing. A file once opened using **`tell()`** will remain opened until we use the **`told()`** predicate.

The see and seen

Like **`tell()`** and **`told()`**, Prolog has **`see()`** and **`seen()`** predicates that are used for reading information from file and not from the keyboard. The **`see()`** predicate therefore help us to change the current input stream. It takes the filename as input. Once the file is read and the read operation is completed, we will use **`seen()`** predicate. The **`seen()`** command transfers the control to the console again to allow accepting input from the console.



Credit: Copyright (C) 1999-2021 Daniel Diaz

The `put(C)` and `put_char(C)` Predicates

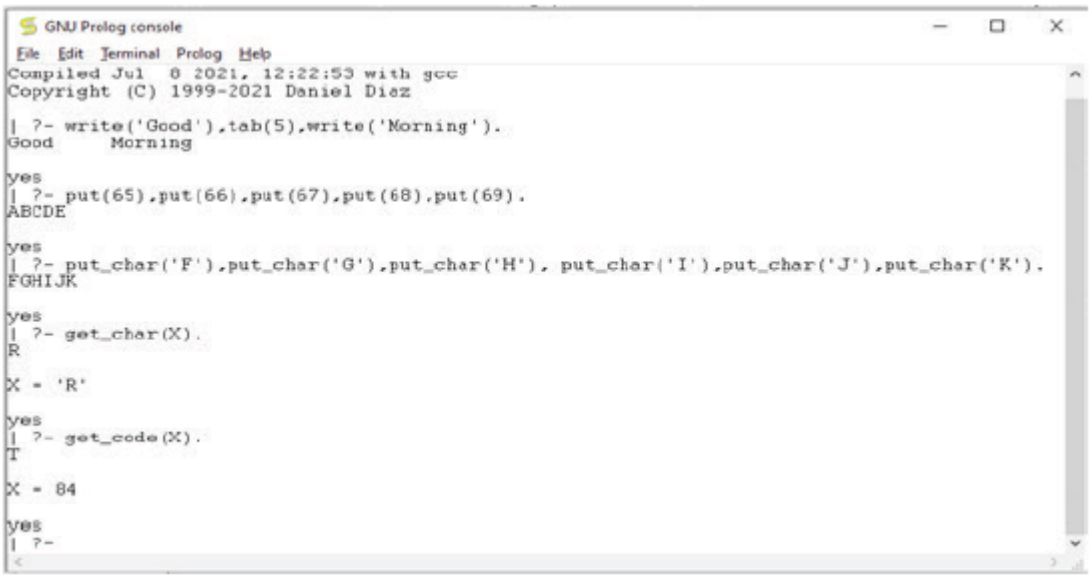
The **`put(C)`** predicate is used to write a character, C at a time in the current output stream which can be either a file or the console.

The **`put_char(C)`** function does the same work but instead of accepting a character, it takes an ASCII code as an input.

The `get_char(C)` and `get_code(C)` Predicates

Like **`put()`** and **`put_char()`**, we have **`get_char()`** and **`get_code()`** predicates in Prolog. As indicated by the name, **`get_char()`** reads a sin-

gle character from the current input stream and the **`get_code(C)`** is used to return the ASCII value of the character, C.



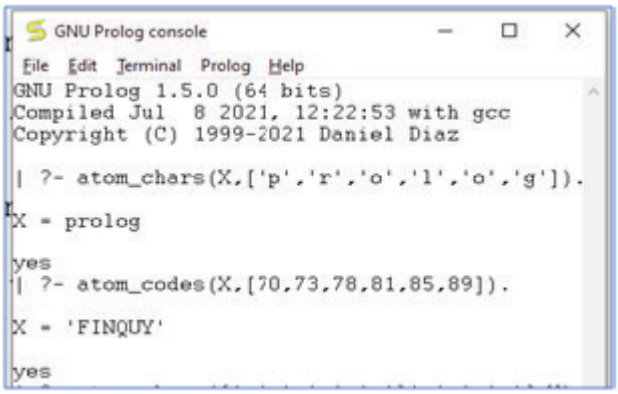
Credit: Copyright (C) 1999-2021 Daniel Diaz

Constructing Atoms

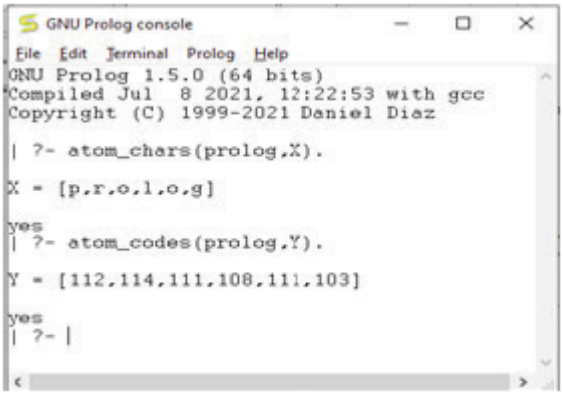
Atom constructing means forming individual atom(s). We get a single atom from a list of characters and multiple atoms a list of ASCII values. **`atom_chars()`** constructs atom from characters and **`atom_codes()`** construct atoms from ASCII values. In both the predicates, the first argument will be one variable, and the second argument will be a list.

Decomposing Atoms

Atom decomposing is just the reverse of constructing atoms. It helps us to get a sequence of characters, or a sequence ASCII codes from a given atom. To do this, we will again use **`atom_chars()`** and **`atom_codes()`** predicates but now, the first argument is an atom and the second argument is a variable. While **`atom_chars()`** decomposes atom to characters, **`atom_codes()`** decomposes them to ASCII values.



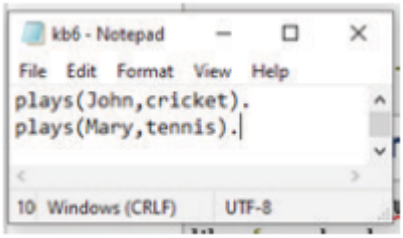
Credit: Copyright (C) 1999-2021 Daniel Diaz



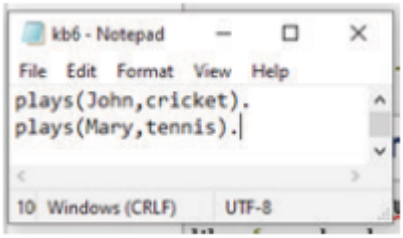
Credit: Copyright (C) 1999-2021 Daniel Diaz

The `consult()` Predicate

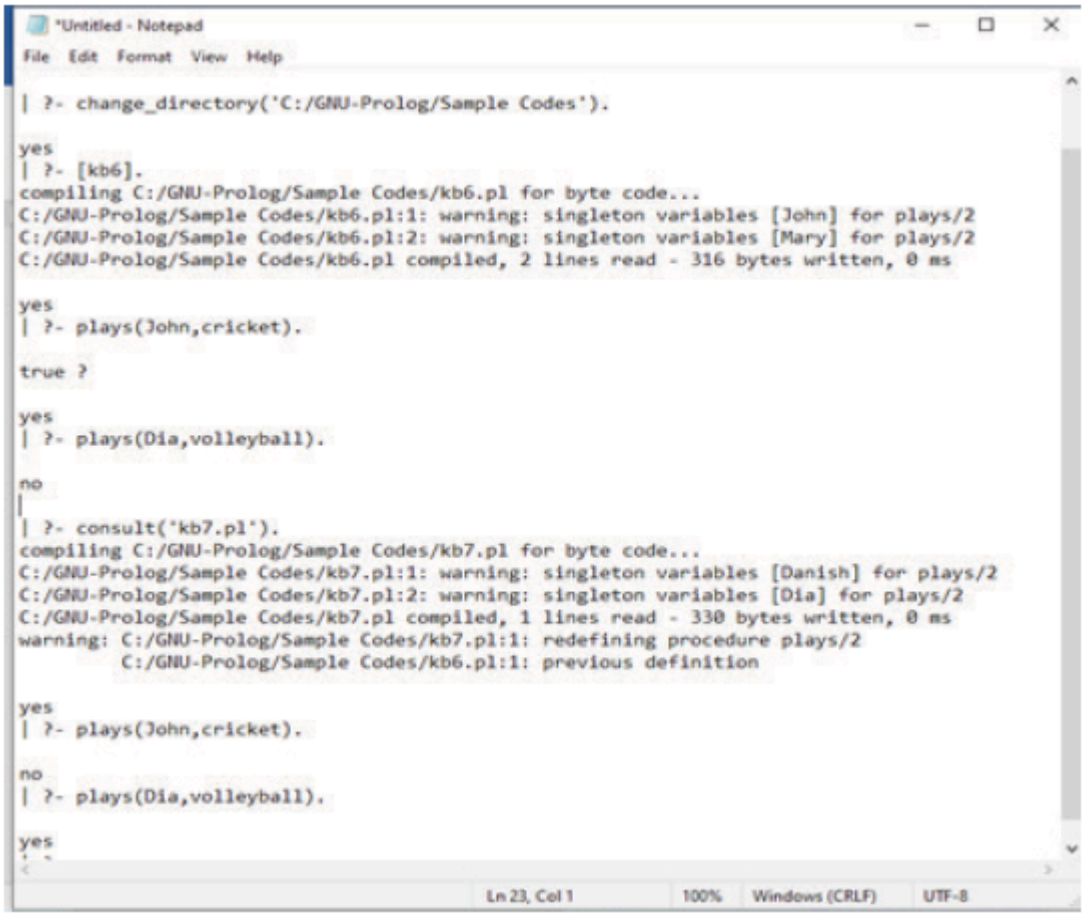
The **`consult()`** predicate is used to merge the predicates from different files. It accepts filename as an argument for consultation. To understand this concept, make two .pl files and execute the following instructions in console.



Credit: Copyright (C) 1999-2021 Daniel Diaz



Credit: Copyright (C) 1999-2021 Daniel Diaz



Credit: Copyright (C) 1999-2021 Daniel Diaz

Now from this output we can understand that this is not as simple as it seems. If two files have completely different clauses, then it will work fine. But if there are same predicates, then while we try to consult the file, it will check the predicates from the second file, when it finds some

match, it simply deletes all of the entry of the same predicates from the local database, then load them again from the second file.